

From the proposed SoftRole-HetNet to the implemented architecture

Design comparison, mathematical rationale, evidence and contribution scope

29 September 2026 · implementation reviewed at commit `7718517` on `softrole`

1. The central change

Your proposal asked whether a **stochastic, persistent, information-regularized role** improves communication and adaptation beyond direct conditioning on history and capabilities. The implemented model first tests that simpler direct conditioning approach. It preserves most of your communication architecture, while removing the Gaussian role process and its KL objective, correcting the team-credit calculation, and tightening the observation and evaluation contracts.

This changes the research question. The implementation cannot support the original claim about the benefits of a rate-limited latent role, because that mechanism is absent. Its question is:

Under a fixed binary communication budget, does a shared policy with capability- and history-conditioned weight mixtures coordinate better than simpler shared policies when team composition changes or a known sensor fails?

The strongest justification for the revision is experimental clarity and a better-specified objective. There is no theorem saying that the deterministic version must outperform your stochastic proposal. Deterministic same-bank conditioning was already one of your proposed controls; I promoted it to the primary model rather than inventing a replacement mechanism.

This document compares three distinct objects: the released HetNet baseline, your supplied SoftRole proposal/patch, and the current implementation. It credits retained proposal ideas explicitly and separates mathematical facts from design choices and empirical hypotheses. The complete runnable protocol remains in [RESEARCH.md](#); implementation checks are recorded in [AGENTS.md](#).

2. Sources and how to read the comparison

The supplied documents are not completely interchangeable:

Source	What it establishes
Architecture, Explained — 27 pages	Intended model, equations, prior/KL argument, limitations and research question. Page references below use the PDF's printed page numbers.

Source	What it establishes
Implementing It, newer version — 39 pages	Repository integration, ablations, environment limitations and the identified observation-aliasing problem.
Implementing It, earlier version — 37 pages	Earlier integration description; it lacks the newer version's full discussion of the observation aliasing issue.
softrole.patch	The proposed executable changes, including stochastic roles, per-agent GAE, group normalization and inherited collection machinery. A documented intention is not necessarily implemented here.
Current model, environment adapter, learner, training engine	What actually runs on this branch.
Released model, binary graph layer, graph construction	Concrete baseline behavior, distinguished from high-level statements in the HetNet paper.

The older local `PLAN.md` is an intermediate critique, not the specification of the implemented deterministic model. No runtime code was changed to produce this comparison document.

3. Three-way architecture comparison

Component	Released HetNet	Your SoftRole proposal and patch	Implemented revision
Agent heterogeneity	Fixed P/A types select graph relations, communication transforms and output heads; some preprocessing is shared	Shared agent network; known own capabilities; Gaussian latent role selects soft weight mixtures	Shared agent network; known own capabilities; deterministic history/capability gate selects mixtures
Environmental observation	Typed P/A occupancy and class-dependent sensing	Patch retains four typed sensory channels; proposal recognizes aliasing bug	Three sensory channels: outside, target and total agent count; both own capability flags appended; opt-in alias correction
Recurrent state	Shared position/status branch plus a P-only sensor branch; no explicit previous received-feature input	One LSTM with previous communicated features and previous action	Retained shared LSTM and feedback; capabilities enter its observation encoder rather than an additional raw-capability bypass
Role/gate	No learned role gate	Gaussian rho in R^3 ; softmax projection to four experts; one role sample per agent-step	Direct MLP gate $g(H, \kappa)$, $66 \rightarrow 32 \rightarrow 4$; one gate per agent-step
Temporal role regularization	None	Capability-resetting autoregressive Gaussian prior and KL penalty	Omitted; temporal dependence comes through the LSTM, without an explicit switching penalty

Component	Released HetNet	Your SoftRole proposal and patch	Implemented revision
Conditional transforms	Class-indexed self/encoder/decoder and relation-specific attention	Convex affine banks; sender gate for encoding, receiver gate for decoding/attention	Retained
Attention normalization	Separate incoming softmax for each source-class relation, followed by summation	One softmax across all received messages plus a zero-message null candidate	Retained; implemented with dense masked tensors
Attention score	Additive GAT within a relation	Receiver-conditioned GATv2	Retained
Binary communication	Active binary layer shares payload across heads; two rounds	Bernoulli-logistic bits with a straight-through backward surrogate; two rounds	Retained; default 16 bits per round, 32 generated payload bits per agent-step
Action head	Separate P/A output heads	Shared six-action head with physical capability mask	Retained
Critic	Released selected path uses a state-summary representation and per-class values; other critic variants exist	State-summary attention plus role/capability-conditioned per-agent values	One permutation-invariant scalar team-value network
Actor credit	Class-based/per-agent reward processing in the selected release path	Per-agent GAE, normalized within capability groups	Common advantage from summed physical-agent rewards; no advantage normalization
Gradient aggregation	Inherited clipping/aggregation/step scaling	Patch reuses that machinery and clips within its policy update	Collect episode-gradient sums; average once by global episode count; clip once
Membership changes	Count-dependent graph/bookkeeping, with class-shared tensors that can transfer	Model-level active masks and birth/death rules; supplied environments do not implement a complete event protocol	Variable roster between episodes; no within-episode arrival/departure implementation
Evaluation claim	Paper includes team-size training/scalability experiments	Proposed frozen transfer, capability events and role interventions	Explicit held-out splits, frozen checkpoints, separate random streams, event-timed gate controls and matched no-failure shams

Several rows change together relative to HetNet. A direct win against HetNet would therefore be evidence about the complete reformulation, not proof that the gate alone caused the improvement.

4. What I retained from your proposal, and why

4.1 Communication enters recurrent memory

Your Architecture §2, pp. 6–8, adds previous postcommunication features and the previous action to the recurrent input. I retained that design:

$$(H_{jt}, C_{jt}) = \text{LSTM}([\text{ReLU}(W_{pre}x_{jt} + b_{pre}), \bar{u}_{j,t-1}, \text{onehot}(a_{j,t-1})], (H_{j,t-1}, C_{j,t-1})).$$

The useful mechanism is explicit: a received message can influence future decisions even when it is not repeated. In released HetNet, communication can still influence later observations through actions and the environment, but it has no direct received-feature path into the next recurrent update. It would be too strong to say that communication has no influence on any future behavior.

One small implementation adjustment deserves disclosure. Your equation also concatenates raw kappa into the LSTM input. Here kappa is already appended to x before the observation encoder; the gate and critic still receive it directly. This avoids a duplicate recurrent input. It is a simpler interface, not an exact reparameterization guarantee or a proven improvement over the raw bypass.

The `--no-feedback` ablation is necessary because a gain could come from message memory rather than conditional weight mixtures. Every carried recurrent tensor is detached at the same five-step boundary during training.

4.2 Convex weight banks and receiver-conditioned decoding

Your Architecture §4, pp. 9–11, defines

$$B(x, g) = \sum_{c=1}^K g_c(W_c x + b_c), \quad g_c \geq 0, \quad \sum_c g_c = 1.$$

I retained it because mixing affine outputs is exactly the same as mixing their weights and biases:

$$B(x, g) = \left(\sum_c g_c W_c \right) x + \sum_c g_c b_c.$$

For any matrix norm, the triangle inequality gives

$$\left\| \sum_c g_c W_c \right\| \leq \sum_c g_c \|W_c\| \leq \max_c \|W_c\|.$$

This bounds the effective matrix relative to the current experts. It does not bound expert growth during optimization. Since g depends on x/history, the full Jacobian also contains terms involving derivatives of g; the inequality is not a global network-stability proof.

Your sender/receiver distinction also remains: the sender's gate selects its encoder; the receiver's gate selects decoding and attention. A receiver can interpret the same broadcast according to its own state without receiving the sender's role vector. HetNet already used receiver-class decoding. The change from HetNet is replacing that discrete class selection with a local soft gate.

A one-hot gate selects one expert in an individual bank, but that does **not** recover the entire original HetNet. Observation information, recurrence, attention normalization, outputs and critic have also changed. Finite softmax logits approach a one-hot gate only as a limit.

4.3 Binary channel, GATv2 and the null candidate

Your Architecture §§5–6, pp. 11–14, already contains all three. Each bit obeys

$$b = \mathbf{1}[u + \ell > 0], \quad \ell \sim \text{Logistic}(0, 1), \quad \Pr(b = 1 \mid u) = \sigma(u).$$

The proof is $\Pr(\ell > -u) = 1 - \sigma(-u) = \sigma(u)$. This is a vector of Bernoulli bits, not one categorical choice among bit positions. Two rounds of 16 bits give 32 generated payload bits per sender-step. Heads share the payload in both the proposal and the active HetNet binary implementation; this is not a newly invented bandwidth reduction. Radio overhead and per-edge deliveries are different accounting measures.

For receiver j , sender k and head q , the retained score has the form

$$e_{jk}^q = a^q(g_j)^T \text{LeakyReLU} \left(A^q(g_j)[z_j^q \| m_{jk}^q] + d^q(g_j) \right).$$

In the original scalar additive score $\text{LeakyReLU}(q_j + k_k)$, monotonicity fixes key ranking within a relation independently of q_j . Applying the nonlinearity before the final vector projection avoids that restriction. This is the expressivity motivation established by [GATv2](#), not evidence that this task's return must improve. Receiver-dependent decoders and banks also change rankings, so isolating GATv2 itself would need another matched ablation.

The null candidate has a learned receiver-conditioned score and a message exactly equal to zero. Consequently an empty neighborhood gives zero received aggregate, regardless of decoder bias. With neighbors, the total real-message attention mass is $1 - \alpha_{\text{null}} \leq 1$. This permits learned attenuation.

There is an important change relative to HetNet: its separately normalized P/A relations each contribute total attention mass one when nonempty, and those aggregates are summed. The single softmax removes that automatic equal mass per source class. It can weigh messages without class labels, but useful minority senders can face dilution in a larger neighborhood. Thus class-label removal changes an inductive bias as well as removing parameters. Neither normalization is uniformly superior.

The dense [receiver, sender] implementation computes the same masked sums as your single-relation graph, with $O(N^2)$ work. That is an engineering simplification for small teams, not a new attention architecture or a large-scale efficiency claim.

5. What I changed, with mathematical reasons

5.1 Promote deterministic conditioning to the main model

Your original role is

$$\rho_{jt} = \mu_{\theta}(H_{jt}, \kappa_{jt}) + \sigma_{\theta}(H_{jt}, \kappa_{jt}) \odot \epsilon_{jt}, \quad \epsilon_{jt} \sim \mathcal{N}(0, I_3), \quad g_{jt} = \text{softmax}(G\rho_{jt} + g_0).$$

The revision is

$$g_{jt} = \text{softmax}(W_2 \text{ReLU}(W_1[H_{jt}, \kappa_{jt}] + b_1) + b_2),$$

with widths $66 \rightarrow 32 \rightarrow 4$. There is no rho, posterior variance, role-prior embedding, stored previous role, or role-sampling noise. Actions and binary messages remain stochastic. Both models reuse one gate across the two rounds.

The decision removes several interacting questions: whether state-dependent weights help, whether latent noise helps, whether a KL penalty helps, and whether temporal regularization helps. First establish the smallest mechanism required by the user's goal. Direct capability-conditioned shared policies have empirical precedent in [Howell et al.](#), while [CASH](#) demonstrates related conditional parameter sharing. These results motivate the experiment; their numerical gains cannot be transferred to this architecture or these observation rules.

This revision is not obtained merely by setting $\beta=0$: that would leave a stochastic role. Nor is a deterministic point mass a harmless zero-variance Gaussian inside the same KL objective; its KL to a nonsingular Gaussian prior is not finite. It is a different model and objective.

Your Implementation (1) already includes deterministic same-bank conditioning as an ablation (§4, pp. 17–18; §10, p. 35). The substantive decision was to make that control the initial research model. The tradeoff is losing an explicit latent information budget and a separately parameterized persistence mechanism. We cannot now claim that either mechanism is useful or unnecessary.

The default PCP dimensions show exactly what was simplified:

Stage	Supplied patch	Implemented model
Observation	726 entries	702 entries
Observation encoder	726→128	702→128
LSTM input	128 encoded + 2 raw capability + 16 feedback + 6 action =152	128 encoded + 16 feedback + 6 action =150

Stage	Supplied patch	Implemented model
Recurrent hidden state	64	64
Role/gate head	66→32→6, split into mean3/variance3, then role3→gate4	66→32→4

The original deterministic control's consecutive affine mean and gate projections can be composed. Removing those intermediate coordinates is not itself evidence of greater expressivity. The full models also differ in their observations, critic and learner, so this is not a claim of whole-model equivalence.

5.2 Defer the persistence prior and KL objective

Your Architecture §§8–10, pp. 15–21, uses the prior

$$p_t = \mathcal{N}(\lambda \rho_{t-1} + (1 - \lambda) \eta(\kappa_t), (1 - \lambda^2) I)$$

with lambda reset to zero on a capability change. Its closed-form Gaussian KL and stationary-prior variance calculation are valid. The reasons for deferral are about interpretation, gradient estimation and experimental scope.

Persistence is not free. With unchanged capabilities and $q_t = \mathcal{N}(\rho_{t-1}, (1 - \lambda^2) I)$, the mean mismatch gives

$$D_{KL}(q_t \| p_t) = \frac{(1 - \lambda)^2}{2(1 - \lambda^2)} \|\rho_{t-1} - \eta(\kappa_t)\|^2.$$

An unchanged posterior mean still costs KL unless it equals the default (or a degenerate boundary case is approached). Resetting the prior changes its reference distribution; it does not force q to match it. The prior's roughly 9.5-step correlation timescale at $\lambda = .9$ is not a measured timescale of the learned posterior trajectory.

The information identity needs a restricted interpretation. Let $c_t = (\rho_{t-1}, \kappa_t)$ and let $\bar{q}$ average the posterior over histories given c . Under the proposal's conditional factorization,

$$\mathbb{E} D_{KL}(q(\rho_t | H_t, \kappa_t) \| p(\rho_t | c_t)) = I(\rho_t; H_t | c_t) + \mathbb{E} D_{KL}(\bar{q}(\rho_t | c_t) \| p(\rho_t | c_t)).$$

Add and subtract $\log \bar{q}$ inside the expectation to obtain the identity. Therefore the logged KL bounds conditional information from above; it includes prior mismatch and is not a direct measure of useful role information or radio bits. No such KL measurement exists in the implemented model.

Analytic KL does not make its expected gradient exact. Consider an earlier action

$a_0 \sim \text{Bernoulli}(\text{sigmoid}(\theta))$, zero task reward and a future cost $K_1 = c * a_0$, with $c > 0$. This can itself be a Gaussian KL: use unit variances, prior mean 0 and posterior mean $\sqrt{2 * c * a_0}$. For objective $-\beta \mathbb{E}[K_1]$,

$$\frac{dJ}{d\theta} = -\beta c \sigma(\theta)(1 - \sigma(\theta)).$$

Differentiating only the realized cost at a fixed sampled state gives zero. Earlier action scores must include their downstream regularization costs, in addition to direct derivatives of those costs. This follows the distinction between sampling and differentiable dependencies formalized by [stochastic computation graphs](#). The patch computes action advantages from environmental rewards and separately differentiates KL; that does not in general implement the stated expected regularized objective.

These are reasons to redesign and validate the stochastic extension before using it, not proof that KL regularization is inherently unsuitable. The proposal's optional MI/discriminator fallback was future work, not an existing patch feature that I removed. I also do not rely on its sweeping criticism of ROMA's identifiability objective to justify this revision; assessing that claim requires its own derivation and implementation audit.

5.3 Remove class labels from observations, not just parameter names

The supplied patch keeps separate P and A occupancy channels. Deleting typed graph nodes alone therefore does not remove direct neighbor-type information. The revised transformation is

$$x_j = [\text{vec}(\text{position}), \text{vec}(s_j[\text{outside}, \text{target}, n_P + n_A]), \kappa_j], \quad \kappa_j = (s_j, a_j).$$

The sensory part is many-to-one: $(n_P, n_A) = (1, 0)$ and $(0, 1)$ produce the same total occupancy. It preserves the count two when agents co-locate. This is a precise reduction in supplied class information. At map width 5 and vision 2, the input has $25 \times (25+3) + 2 = 702$ entries, versus 726 in the supplied patch. Both capability flags belong to the agent itself; neighbor capabilities must be inferred or communicated.

This does **not** mean statistical independence from class. In the original team, $(1, 0)$ and $(0, 1)$ largely identify the two physical types. The defensible claim is no explicit class label, class-indexed actor weights, or direct typed neighbor-occupancy channel. The simulator still needs physical type bookkeeping. The design is capability-conditioned, not capability-agnostic or unsupervised fault diagnosis.

The newer supplied implementation guide already identifies the observation-view aliasing problem (§7, pp. 27–28), but the patch does not apply the fix. The new adapter opts into copied observation windows so blinding one window cannot mutate another agent's input. I implemented an existing diagnosis; I did not discover a new theoretical mechanism. Original reproduction entrypoints retain their default behavior, and the corrected path has an explicit version tag.

This change also makes comparisons harder: typed HetNet has richer information, and an original checkpoint evaluated on corrected windows experiences an observation shift. That reference is

informative but cannot isolate the causal benefit of conditional weights.

5.4 Align actor credit and the critic with the team objective

The Architecture §9, pp. 17–18, states a summed team objective, while its update and the patch use each agent's own GAE and normalize within capability groups. Those are generally different objectives in a cooperative task.

For the task objective

$$J = \mathbb{E} \sum_{t=0}^{\tau-1} r_t^{team}, \quad r_t^{team} = \sum_j r_{jt},$$

the joint action score sums agent log-probability scores and gives each credit for downstream team reward. The distinction is visible in a one-step example:

$$a_1 \sim \text{Bernoulli}(\sigma(\theta)), \quad r_1 = 0, \quad r_2 = a_1.$$

The team gradient is $\sigma(\theta) * (1 - \sigma(\theta))$; the own-reward update for agent1 is zero. Differentiable communication paths do not restore every omitted effect of a sampled physical action on teammates' rewards.

The revised learner uses one common team advantage and one team-value target. The critic is

$$V_t = \psi \left[\frac{1}{N} \sum_j \phi([H_{jt}, \kappa_{jt}]), N, \sum_j s_{jt}, \sum_j a_{jt}, (T - t)/T \right].$$

This replaces the proposal's attention-based state-summary node and per-agent role-conditioned values. Permuting agent order leaves the mean and counts unchanged. The form is consistent with the invariant aggregation principle in [Deep Sets](#); the exact finite network used here is not claimed to inherit an unrestricted representation theorem.

A centralized baseline computed before current action sampling satisfies $\mathbb{E}_a[b(h) * \text{grad} \log \pi(a|h)] = b(h) * \text{grad} \sum_a \pi(a|h) = 0$, with its gradient detached in the score loss. Centralized critic inputs never feed the actor forward pass. That separation was already in your proposal and HetNet; it is retained. Value loss still changes the shared encoder/LSTM, so actor representation learning is not isolated from the critic.

Using a simpler invariant critic aligns the prediction target with the new team objective and removes an extra learned attention mechanism. It may sacrifice useful critic capacity. No evidence yet establishes that mean pooling is a better value estimator than the proposed state-summary attention.

5.5 Make the update's weighting explicit

The implemented episode loss is

$$L_e = -50 \sum_t \left(\sum_j \log \pi_j(a_{jt} \mid \mathcal{I}_{jt}) \right) \text{sg}(\hat{A}_t^{team}) + \frac{1}{\tau_e} \sum_t (V_t - \hat{R}_t)^2.$$

Here $\gamma=1$, $\lambda_{\text{GAE}}=.95$, terminal $V_{\text{tau}}=0$, and $\text{Rhat}=\text{stop_gradient}(\hat{A}+V)$. The critic uses actual remaining time because the horizon is the specified task deadline. The actor sums time terms; the value loss averages over time. There is no extra normalization by agents or steps.

Normalizing advantages separately for each capability group introduces separate data-dependent offsets and scales. Even ignoring finite-batch dependencies, different group scales change their relative contribution; this is not merely a single common learning-rate adjustment. Removing group normalization makes the chosen reward weighting easier to audit. It may increase gradient-scale variation and is not a general argument against all advantage normalization.

For worker w with E_w episodes, let G_w be the sum of its episode gradients. The desired empirical average is

$$G = \frac{\sum_w G_w}{\sum_w E_w}.$$

For example, $E_1=1$, $G_1=2$, $E_2=3$, $G_2=12$ gives $G=3.5$; averaging the two worker means gives 3. Clipping is nonlinear, so clipping each worker first can also change direction after aggregation. The new engine averages once, clips once at norm .75, then performs one RMSprop update. It keeps $\text{lr}=1\text{e-}4$, $\alpha=.97$ and $\text{eps}=1\text{e-}6$. These hyperparameters and the coefficient 50 are protocol choices, not mathematical optima.

This corrects weighting relative to our declared objective. It changes the original optimizer pipeline and must be held constant across the new ablations. The original HetNet learner should not be called universally incorrect: its own-agent/group processing is simply not, in general, the gradient of this explicit summed-team objective.

5.6 Keep approximation claims honest

The implemented gradient is still not an unbiased exact gradient of expected team return. The main approximations are:

Approximation	Why it matters
Straight-through binary messages	The hard forward channel has discrete sampling dependencies; the backward path substitutes a relaxed derivative.
GAE with $\lambda=.95$ and an approximate critic	Reduces variance at the cost of possible bias; see Schulman et al. .

Approximation	Why it matters
Five-step recurrent detachment	Removes parameter-gradient paths through longer histories while retaining their numerical states.
Learned shared actor/critic representation and finite-sample optimization	Add practical coupling and estimation error; algebraic checks do not prove convergence.

The binary bias can be shown without a complicated environment. Let reward be the single bit b . The true derivative is $d E[b]/du = p(1-p)$, which equals $1/4$ at $u=0$. The temperature-1 surrogate derivative at $u=0$ is $\sigma(\ell)(1-\sigma(\ell))$. Since $U=\sigma(\ell)$ is uniform on $(0,1)$,

$$\mathbb{E}[U(1-U)] = \int_0^1 (u - u^2) du = 1/6 \neq 1/4.$$

Thus correcting team credit does not eliminate message-estimator bias. This choice retains the proposal's practical low-variance surrogate, accompanied by an exact discrete reference test, rather than asserting a proof it does not have.

6. How adaptation and team changes are actually represented

The mechanism after a sensing loss is concrete. Before the scheduled action, the victim's sensing flag becomes 0 and its current environmental sensory channels are masked. Position information, recurrent memory and communication remain available. The gate can change immediately using its own capability and new hidden state. Its payload and action can therefore change. Other agents do not receive an oracle list of failed teammates; they can react through messages and observations. Their gate can incorporate newly received information through the next recurrent update, while their current action can already use this step's received features.

This is a mechanism that **permits** adaptation, not evidence that training has learned the right response. In particular:

- Own capability is supplied. Unknown, delayed or incorrect failure detection is a different problem.
- The failed agent remains present. Memory of pre-failure observations is not erased; a sensor failure is not total amnesia or loss of localization.
- The code supports episode-level changes in team size. It does not currently implement mid-episode births, removals or reward reassignment after departure. The original guide already acknowledged missing simulator lifecycle support; its active masks did not constitute a complete arrival/departure experiment.
- Failures are implemented only for PCP. In FC, sensing updates internal fire discovery state, so output masking alone would not correctly remove sensing effects from the physical process. PP/PCP/FC domain execution is supported, but FC sensor-failure robustness is not a current claim.

The reason the network can execute at different N is that every local map has fixed feature dimensions and aggregation operates over a set. For a permutation matrix P and correspondingly permuted capabilities, memory, adjacency and noise, shared maps and neighborhood sums give

$$F(PX, P\kappa, PMP^T; P_{\text{memory}}, P_{\text{noise}}) = PF(X, \kappa, M; \text{memory}, \text{noise})$$

for actor outputs. The pooled critic is invariant. This is an inductive argument through recurrent steps and communication rounds. With independent fresh noise, the policy is equivariant in distribution; pathwise equality requires coupled noise. It does not imply high return on unseen N, new spatial densities or unseen capability combinations.

Original HetNet also has class-shared parameter tensors whose shapes can support different counts. Our strict-load tests verify unchanged-weight execution from 2P1A to 3P2A and 1P2A for Real and Binary models, with a narrow singleton shape repair. The original paper's §6.3.4/Fig.5c reports training/scalability at several compositions; that is different from the fixed-checkpoint held-out protocol used here. Neither fact proves successful frozen generalization. [HetNet paper](#).

7. Evaluation changes and what they let us conclude

Your implementation guide already proposed composition splits, frozen interventions, multiple seeds and uncertainty reporting (§10, pp.34–35). The revision makes these intentions operational and changes some choices:

Adjustment	Reason and inference limit
Explicit composition list rather than rectangular count ranges	Prevents a designated held-out combination from appearing accidentally during training.
Training compositions (1,1),(1,2),(2,1), (2,2),(3,1); held-out (3,2), with (4,1)/(4,2) additional cases	The primary held-out team combines previously seen marginal P/A counts, but also increases total N from at most 4 to 5. It therefore tests composition and size transfer together. (4,1) introduces an unseen P count at N=5; (4,2) also increases N to 6.
Failure training restricted to (2,1),(2,2), (3,1), .5 event probability, random step 10–30 and random sensing victim	Includes no-event experience and leaves at least one working sensor. This avoids complete team sensing loss, but severity still differs: 2P loses half its sensors, while 3P loses a third. Report each composition separately.
Freeze affected/all gates or remove received communication starting at the scheduled event	Preserves the pre-event prefix. Freezing the gate does not freeze the recurrent state or entire policy.
Matched —sham controls	Retains event time, victim and random streams while suppressing actual failure, testing whether gate freezing is harmful even without sensor loss.
Separate environment, action and message randomness	Policy/intervention-dependent random consumption cannot shift another episode's initialization or later time-indexed draws. Actions may still create different subsequent states.
Explicit event exposure, early completion and censoring	A task that finishes before the event is not a post-failure recovery. Averaging only successful recovery times would omit failures.

Adjustment	Reason and inference limit
Frozen checkpoint signatures and complete scenario records	Establishes which weights and assigned scenarios were evaluated.
Equal-weight training-seed means and whole-seed bootstrap	Evaluation episodes are not independent trained models. More episodes do not substitute for independent training seeds.

The current summary uses mean success/return and horizon-capped completion time; it does not implement the original guide's IQM prescription or a paired inferential comparison. The bootstrap is conditional on the supplied evaluation panel and is unstable with very few training seeds. The importance of independent runs and uncertainty is supported by [Agarwal et al.](#). A single seed receives no between-training-seed confidence interval.

A paired adaptation analysis can later compare, within the same seed/scenario, the loss from freezing under failure with the loss from freezing under sham:

$$D = (Y_{adapt,failure} - Y_{freeze,failure}) - (Y_{adapt,sham} - Y_{freeze,sham}).$$

For an outcome where larger is better, positive D suggests gate updates are especially useful under failure, beyond their usefulness in ordinary operation. It remains a within-model intervention on a possibly unusual gate trajectory, not proof of semantic role discovery. The code supplies the matched outcomes; it does not currently calculate or test D. Use the complete assigned panel for the primary contrast; conditioning on event exposure can select different subsets if policies finish at different times.

The necessary controlled baselines

Model	What it tests	Current default PCP parameters
shared	Is a capability-aware recurrent communication policy sufficient without banks?	171,087
constant	Do adaptive gates help beyond the optimization/capacity of a fixed bank mixture?	221,899
capability	Does the gate need history beyond current own capabilities?	222,123
banked	Full history/capability-conditioned gate	224,171
banked --no--feedback	Does carrying previous communicated features matter?	Count differs from full banked

These models use the same new information contract and learner. They are not all parameter-matched. A fixed convex mixture is functionally an affine map in each bank, although its overparameterized optimization differs from a single expert. The smaller shared baseline and larger conditional banks answer a practical comparison; a strict parameter-budget claim needs additional matched controls.

An original HetNet comparison changes observation information, normalization, recurrence, binary parameterization, critic and optimization simultaneously. Consequently it is a contextual reference. Matching epochs alone is insufficient: report actual environment steps, parameters and compute. The new package keeps the released PP/PCP/FC domain recipes available, but this does not mean it exactly reproduces original-paper scores or constitutes a pure architecture-only ablation.

8. Research evidence: what it supports and what it does not

The cited results supply precedents and mathematical tools. None evaluates this exact implementation, so none is empirical proof of its performance.

Primary source	Relevant evidence	Boundary on our inference
Seraj et al., HetNet, AAMAS2022 , §§4–6	Class-indexed heterogeneous communication, binary messages, critic alternatives and domain evaluations	Establishes the baseline and inherited ideas; does not establish this revision's advantage.
Howell et al., CoRL2023	Empirical adaptive teaming through capability awareness and communication in shared decentralized policies	Supports studying explicit capability inputs; prevents claiming that capability-aware shared teaming is new.
Fu et al., CASH, arXiv2501.06058v4 , §4 and experiments	Capability/context-conditioned parameter generation and online capability/team changes	Broad conditional-weight adaptation already exists. CASH's documented teammate-capability inputs differ from our bounded-message information contract; performance numbers cannot be compared without matching that contract.
Wang et al., ROMA, ICML2020	Stochastic role embeddings, role-conditioned policies and empirical role-learning results	Motivates the original latent proposal. It does not imply our deterministic gates learn interpretable roles.
Brody et al., GATv2, ICLR2022	Static-ranking limitation of standard GAT and a more expressive attention construction	Supports expressivity choice, not guaranteed improved MARL return or a unique explanation of adaptation.
Schulman et al., Stochastic Computation Graphs, NeurIPS2015	Gradient accounting for stochastic and differentiable dependencies	Supports the action-score/direct-cost distinction and cautions about substituting straight-through derivatives.
Schulman et al., GAE	Advantage-estimator construction and bias/variance tradeoff	Supports an estimator choice; does not prove our complete learning algorithm unbiased.
Zaheer et al., Deep Sets, NeurIPS2017	Permutation-invariant set aggregation	Motivates the critic form; generalization and finite-network sufficiency remain empirical.
Agarwal et al., Statistical Precipice, NeurIPS2021	Reliable evaluation under few-run uncertainty	Supports reporting independent-run uncertainty; a particular small-seed bootstrap is not automatically reliable.

This is a focused evidence review, not an exhaustive novelty search. The proposal mentions additional recent methods; this document does not use unverified names or abstract-level similarity to justify a claim of priority.

9. The contributions we can reasonably claim

Implemented design and methodological contributions

1. **A capability-conditioned reformulation of HetNet's communication pathway.** The implementation replaces fixed class selection with shared, locally conditioned mixtures and removes explicit actor class labels and typed neighbor occupancy. It preserves a fixed two-round binary payload budget. Most of the communication design comes from your proposal; the revision promotes its simpler deterministic control and tightens the information rule.
2. **An objective-consistent team-learning implementation.** The actor score uses common team credit; the critic predicts team value; collector averaging is explicit. This is methodological correctness for the selected objective, not a newly invented policy-gradient theorem or a claim of unbiased training.
3. **A reproducible protocol for frozen composition transfer and known sensing loss.** Disjoint splits, versioned observations, matched event/sham controls, immutable weights and seed-level reporting make the adaptation question falsifiable. This is a study contribution rather than a new environment class.

Empirical contributions that remain conditional on results

The eventual work can report whether the reformulation retains fixed-team performance, whether it improves frozen held-out/failure outcomes, and whether history-conditioned gates or communication memory explain those effects under the controls. These are hypotheses today. If the shared or capability-only model matches the full model, that is a useful finding about which complexity was unnecessary in the tested setting.

The following claims would currently be unsupported:

- First heterogeneous policy without class labels or first capability-aware conditional parameter-sharing method.
- First HetNet-like network able to execute at a new team count.
- Lower payload bandwidth than the corresponding HetNet-Binary configuration.
- Guaranteed fault tolerance, successful size generalization, or convergence.
- Discovery of discrete semantic roles, task assignments or a minimal role code.
- A demonstrated benefit from stochastic roles, KL rate control or persistence.
- Superiority to HetNet, ROMA, CASH or other methods without matched experiments.

Gate values are not probabilities of human-named roles. Expert labels can be permuted together with gate coordinates without changing the function. Gate entropy, expert load and attention plots therefore need interventions and task outcomes to support behavioral interpretations. They are useful diagnostics, not explanations by themselves.

Suggested thesis or paper wording

We investigate a capability-conditioned reformulation of HetNet for heterogeneous cooperative teams. A shared recurrent policy uses deterministic mixtures of communication transforms and two rounds of binary messaging, without explicit actor class labels or typed neighbor-occupancy inputs. We evaluate frozen policies on held-out team compositions and known local sensing failures, using controlled gate and communication interventions to test which mechanisms support adaptation.

This is accurate before obtaining positive results. Add outcome claims only after measuring them. For a graduate-school research portfolio, the defensible value is a clear hypothesis, technically careful implementation, credible controls, and conclusions proportional to the evidence; a claim of unprecedented novelty is unnecessary.

10. Code traceability and present evidence level

Claim or decision	Current implementation	Existing check
Untyped sensory counts, current own capability, corrected views	env.py ; opt-in paths in the original PCP/FC simulators	Observation/environment tests
Convex banks, receiver decoding, null attention and fixed payload	model.py	Model identity, binary, mask and directed-neighbor tests
Equivariance, recurrent feedback and actor/critic isolation	model.py	Coupled-noise permutation, feedback/detach and critic-independence tests in the same file
Common team GAE and approximate message learning	learning.py , rollout.py	Enumerated reward-gradient and binary-surrogate references
Global episode averaging and reproducible resume	train.py	Unequal-worker averaging, spawn/serial and exact resume tests
Frozen transfer, event timing, shams and censoring	evaluate.py , hetnet.py	Evaluation tests , original HetNet transfer tests
Independent-seed summaries	report.py	Weighting, duplicate, stratum and interval tests

The implementation record reports 129 passing tests, twelve short actual training checks across the three domains/two main models/one or four collectors, and failure-versus-sham evaluation checks. Those checks establish specific algebraic and execution properties. They do not constitute a convergence study or an estimate of learned robustness. No new training experiment was run for this document, and no numerical performance improvement is being claimed.